

Performing Garbage Collection in Java

In programming, garbage collection is a form of automatic memory management, whereby the memory used by an object is freed or released by destroying the object when it becomes redundant in the program. Garbage collection is considered one of the best practices to avoid any untoward results when a program is executed.



In object-oriented programming languages (like C++ and Java), there could be unused objects that are still a part of the program. These will create undesirable results like an out of memory error, for instance, during runtime, if left unattended. The memory occupied by such objects is known as 'garbage' and the mechanism to eliminate this from the program is called 'garbage collection'.

In languages like C++ (which is one of the older OOP languages), the programmer is responsible for creating and destroying the objects. If the purpose for which the object was created is fulfilled but the programmer neglects to delete the now-useless objects, the entire memory will be filled with old and unwanted objects. This will prevent the creation of new objects, thereby bringing down the entire application.

Garbage Collection (GC) in Java

Unlike C++, Java provides the programmer with an assistant to delete useless objects, which is called the garbage collector. In Java, there is no *OutOfMemoryError* thanks to the garbage collector, which destroys unused objects. Java is also called a robust programming language because of such mechanisms. Because of this background mechanism, the stress on the programmer is significantly less.

The various ways to make an object eligible for GC

In Java, even though the programmer is not responsible for destroying the objects, it is still highly recommended

that you make an object eligible for garbage collection if it is no longer required. Thereby, the garbage collector will take care of this object as a part of the routine clean-up. For example, if you put a used tea-cup in a garbage bin, the local cleaning authority will naturally take care of it. In Java, when the object does not have any reference pointing to it, then it becomes eligible for garbage collection. The subsequent sections of this article list various ways to make an object eligible for GC.

Illustration 1- Nullifying the reference variable

Let us look at the following code snippet.

```
Student s1 = new Student();
Student s2 = new Student();
s1=null; // One object eligible for garbage collection
```

In the above code, we can see two objects of *Student* (*s1* and *s2*). When we create a second object of the same class, it is obvious that you don't need the first object or the previous object; in which case, it is highly recommended that the previous object is made eligible for garbage collection. So we use *s1=null* to make the previous object eligible for garbage collection.

Illustration 2- Re-assigning the reference variable

Let us take a look at another snippet shown below.